Introduction to Programming

Lecture 11

VU
Virtual University

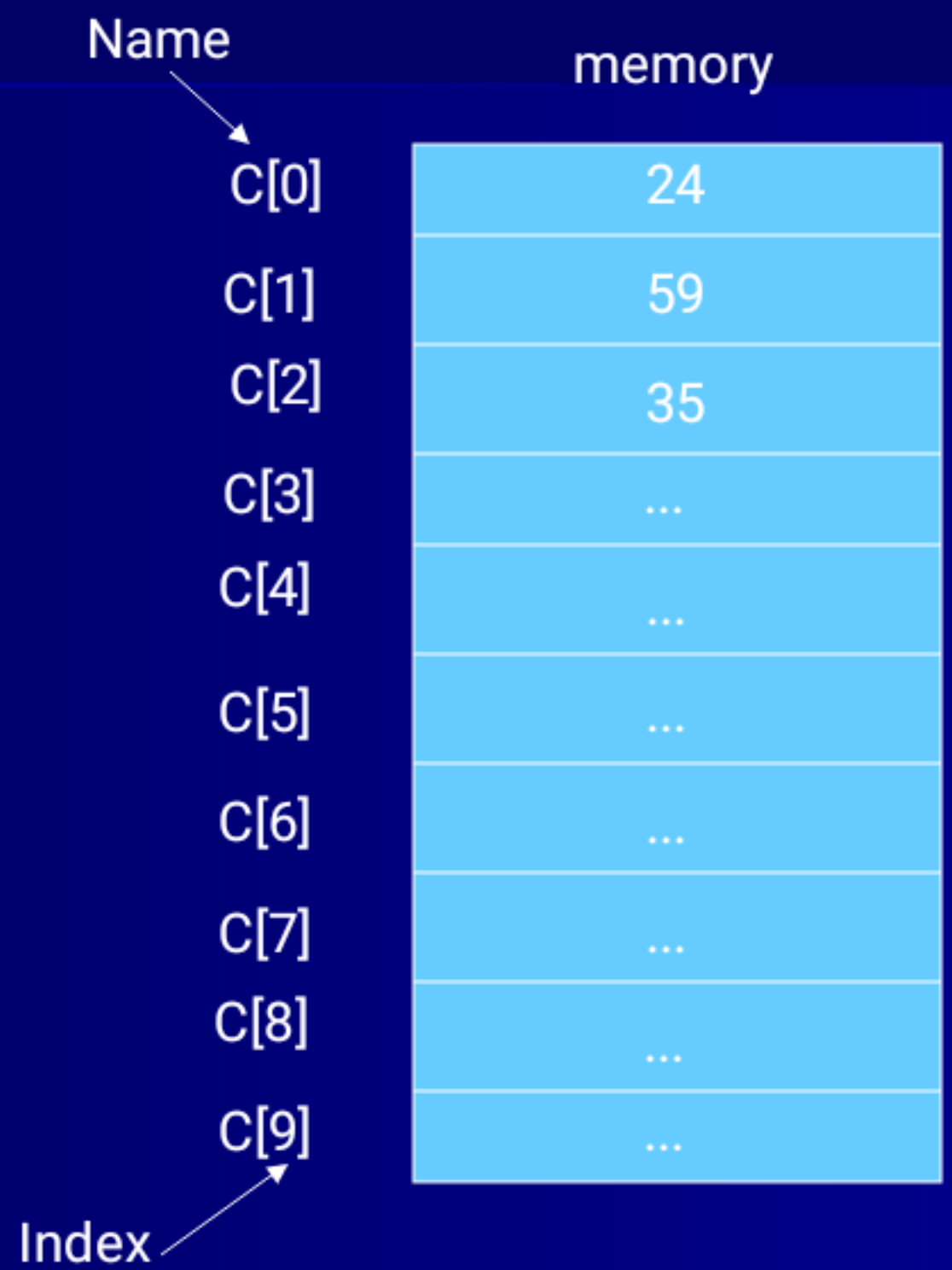
ARRAYS

VU
Virtual University

Arrays

- They are special kind of data type
- They are like data structures in which identical data types are stored
- In C each array has
  - name
  - data type
  - size
- They occupy continuous area of memory

Storage of an array in memory



Declaration of Arrays

arrayType arrayName[numberOfElements];

For example ,

int age [10] ;

- More than one array can be declared on a line

  int age [10] , height [10] , names [20] ;

- Mix declaration of variables with declaration of arrays

  int i , j , age [10] ;

Referring to Array Elements

Array name e.g. age

index number

age [4]

Example1: Using Arrays

```
for ( i = 0 ; i < 10 ; i ++ )
  {
   cin >> age [ i ] ;
  }
```

Example 2

```
totalAge = 0 ;
for ( i = 0 ; i < 10 ; i ++ )
  {
  totalAge + = age [ i ] ;
}
```

Initializing an Array

```
int age [ 10 ] ;

for ( i = 0 ; i < 10 ; i ++ )
   {
 age [ i ] = 0 ;
}
```

Initializing an Array

```
int age [ 10 ] = { 0,0,0,0,0,0,0,0,0,0 } ;
```

```
int age[ 10 ] = { 0 } ;
```

Initializing an Array

```
int age [ ] = { 1,2,3,4,5,6,7,8,9,10 } ;

for ( i = 0 ; i < 10 ; i ++ )
```

‘ i ‘ will have value from 0 to 9

Example: 3

```
#include < iostream.h >
main ( )
{
  int  c [ 100 ] ;
```

Example: 3

```
int z , i = 0 ;
  do
{
 cin >> z ;
 if ( z != -1 )
 c[ i ] = z ;
```

assignment statement

Example 3

```
i ++ ;
} while ( z != -1 && i < 100 ) ;
cout << " The total number of positive integers
   entered by user is " << i -1;
}
```

Copying Arrays

- Data types should be identical
- Size should be same

```
int a [ 10 ] ;
int b [ 10 ] ;
```

Copying Arrays

To copy from array " a " to array " b " :

```
b[0]=a[0];
b[1]=a[1];
b[2]=a[2];
b[3]=a[3];
… … …
… … …
b[10]=a[10];
```

Copying Arrays

```
for ( i =0 ; i < 10 ; i ++ )
    b [ i ] = a [ i ] ;
```

Example: 4

Take the sum of squares of 10 different numbers which are stored in an array

```
int a [ 10 ] ;
int arraySize =10 ;
int sumOfSquares = 0 ;
for ( i = 0 ; i < arraySize ; i ++ )
{
 sumOfSquares = sumOfSquares + a [ i ] * a [ i ] ;
}
```

Example 5

```
int z ;
int a [ 100 ] ;
for ( i = 0 ; i < 100 ; i ++ )
{
 a [ i ] = i ;
}
cout << " Please enter a positive integer " ;
cin >> z ;
int found = 0 ;
```

Example 5

```
for ( i =0 ; i < 100 ; i ++ )
{
    if ( z == a [ i ] )
    {
     found = 1 ;
     break ;
    }
}
```

Example 5

```
if ( found == 1 )
 cout << " We found the integer at position " << i ;
else
 cout << " The number was not found" ;
```

rand ( )

include < stdlib.h >

0 - 32767

Calling rand ( )

x = rand ( ) ;

A call goes to ” rand ( ) “ , it generates a number and returns to x

Modulus " % "

It returns the remainder

rand ( ) % 6 = ?

Result has to be between 0 and 5 inclusive

1 + rand ( ) % 6

It will randomly generate number between 1 and 6

VU Virtual University

Fair Die

If a die is rolled 10/100 million of time , then on average equal number of 1’s ,equal number of 2’s , equal number of 3’s etc. will be generated

Example: Tossing a Coin

It has only two possibilities 0 / 1

```
rand ( ) % 2 ;
```

Importance of rand ( )

- It is shipped in every standard library with compiler
- Most major programming languages give some kind of random number generator as a function as part of library
- Writing a random number generator is itself a field

Array Declaration

data type

name

size

const

const

```
const int arraySize = 100 ;
```

- It creates an identifier " arraySize " and assigns a value 100. This is called integer constant . It is **not** a variable
- Its value cannot be changed